

Clinical Treatment Outcome Project

Updated Business-Oriented Phase Report

Date: 8 September 2026 | **Status:** Modeling, explainability, SQL analysis, and API deployment completed

Executive Summary

The project transformed a large clinical dataset into a cleaned, feature-engineered foundation for treatment-outcome prediction. Decision Tree, Random Forest, XGBoost, and Neural Network models are evaluated on the same 60% training, 20% validation, and 20% test split. The LSTM is evaluated separately on patients with repeated visits because it uses sequential information. Model selection emphasizes precision, recall, F1, and ROC-AUC rather than accuracy alone because the outcome classes are imbalanced.

Twelve-Step Delivery

Step 1: EDA Reviewed structure, missingness, duplicates, distributions, correlations, clinical exceptions, and outcome balance.	Step 2: Cleaning Standardized headers, removed five high-missingness columns, imputed values, removed 70,017 duplicates, and capped outliers.
Step 3: Feature Engineering Created kidney, liver, polypharmacy, BMI, age, dose, AST/ALT, drug-kidney, treatment-duration, and admission-time features.	Step 4: Decision Tree Class-weighted tree trained with encoded categorical data and 60/20/20 performance checks.
Step 5: Random Forest 200-tree class-weighted ensemble evaluated on the same split and metrics.	Step 6: XGBoost Boosted model with imbalance handling and cross-validated tuning; performance checked against the other models.
Step 7: Neural Network Dense neural network trained with the same row-based test split and class weighting.	Step 8: LSTM Separate sequential model trained on up to four chronological visits for patients with repeated records.
Step 9: Model Comparison Shared-test-set models are compared separately from the repeated-visit LSTM evaluation.	Step 10: Explainability SHAP global and patient-level explanations plus LIME patient-level explanations generated for XGBoost.
Step 11: SQL Analysis Six SQLite analyses completed after normalizing treatment outcomes and adverse events to numeric 0/1 values.	Step 12: API Deployment Flask API loads the Step 6 pipeline and passes raw inputs through its fitted preprocessing and XGBoost stages.

Model Results: Shared Test Set

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Decision Tree	0.5798	0.2859	0.6694	0.4006	0.6540
Random Forest	0.6114	0.2860	0.5698	0.3809	0.6326
XGBoost (Selected Best Model)	0.6126	0.2957	0.6127	0.3989	0.6549
Neural Network	0.5744	0.2818	0.6644	0.3957	0.6468

Interpretation: XGBoost is selected as the best deployment model using a composite score weighted toward ROC-AUC (40%), precision (30%), F1 (20%), and accuracy (10%). It leads the shared-test-set models on ROC-AUC (0.6549) and precision (0.2957). Decision Tree has the highest recall (0.6694), so it remains a useful high-sensitivity alternative. Validation-based threshold selection was used for the tuned XGBoost result.

Separate LSTM Evaluation

Model	Dataset	Accuracy	Precision	Recall	F1	ROC-AUC
LSTM	Repeated-visit patients	0.6133	0.3008	0.6208	0.4053	0.6509

The LSTM result is not ranked directly against the row-based models because it uses a different patient population and sequential input structure. Its sequences use up to four chronological visits, with patient-level train, validation, and test separation.

Explainability, SQL, and API

SHAP and LIME explanations were generated from the saved Step 6 XGBoost pipeline using the matching 216,997-row test split; a 500-row sample was explained to control memory use. The SQL workflow normalizes *treatment_outcome* and *adverse_event* to numeric 0/1 values. Q4 compares elderly and non-elderly ADR rates by drug, requires at least 100 patients in each group, and ranks the rate difference in percentage points. The Flask API accepts raw model feature values and delegates preprocessing to the saved pipeline, avoiding double encoding or scaling.

Key Findings

Model performance: XGBoost is the selected deployment model because it leads the shared-test-set models on ROC-AUC and precision and has the highest composite selection score. The LSTM is not directly ranked with these models because it uses a different repeated-visit population.

Patient outcomes: After valid binary normalization, 20.98% of records were effective and 79.02% were ineffective. Class imbalance makes accuracy alone insufficient for model selection.

Age pattern: Treatment efficacy was highest for patients under 30 at 36.04% and declined across age groups to 12.74% for patients over 65. This is an association, not evidence that age causes treatment failure.

ADR pattern: Q4 compares drugs with at least 100 elderly and 100 non-elderly patients. Ciprofloxacin had the largest observed elderly/non-elderly ADR rate difference at 8.07 percentage points, 29.25% versus 21.18%.

Dosage pattern: Efficacy was similar across dosage groups, from 20.94% for low dosage to 21.08% for high dosage. ADR rates were also similar, so dosage alone is not a strong descriptive separator in this dataset.

Explainability and Deployment Controls

SHAP global and patient-level explanations and LIME patient-level explanations were generated for the fitted Step 6 XGBoost pipeline. The Flask API accepts the 41 raw feature-engineered inputs and passes them directly to the saved pipeline, which performs numeric processing, categorical encoding, and XGBoost prediction exactly once.

Limitations and Recommendations

- The target and adverse-event fields required normalization because the source contained mixed labels such as 0, 1, Yes, No, Effective, and Unknown.
- Model precision is modest because the effective class is the minority class; calibration and threshold choice should be reviewed before operational decisions.
- The drug and age findings are observational associations and should not be interpreted as causal treatment or safety conclusions.
- Validate performance by subgroup, time period, and patient level; monitor data drift and retain clinician review for high-risk cases.

- Use the XGBoost API as decision support, not as an autonomous clinical decision-maker.

Business Implications and Next Actions

The workflow can support treatment-review prioritization, risk stratification, explainable model review, SQL-based cohort analysis, and API integration. Before operational use, validate target mappings, investigate repeated patient IDs and invalid dates, complete patient-level and subgroup validation, assess calibration, monitor drift, and retain human oversight.

Conclusion: The project has delivered a measurable, interpretable foundation for treatment-outcome analytics. Decisions should prioritize clinically meaningful error trade-offs over accuracy alone.